

CC3092 – Deep Learning

Hoja de Trabajo 2

Instrucciones

- Esta es una actividad en grupos de no más de 3 integrantes.
 - Recuerden **unirse al grupo de canvas**
- No se permitirá ni se aceptará cualquier indicio de copia. De presentarse, se procederá según el reglamento correspondiente.
- Tendrán hasta el día indicado en Canvas.
 - No se confíen, aprovechen el tiempo en clase para entender todos los ejercicios y avanzar lo más posible.
- **NOTA:** Limiten el uso de IA generativa. Intenten primero buscar en fuentes de internet y si en verdad necesitan usarla, asegúrense de **colcar el prompt** que utilizan para cada task donde corresponda, así como una explicación de por qué ese prompt funcionó.

En esta hoja de trabajo usted va a implementar una DCGAN desde cero con PyTorch para generar sprites de Pokémon. El dataset se descarga directamente desde la API pública de PokeAPI en GitHub: 898 imágenes de 64×64 píxeles RGB que cubren todas las generaciones. A diferencia de los laboratorios anteriores donde el backward era manual, aquí usará `.backward()` de PyTorch, el foco está en entender el juego adversarial, analizar sus fallas, e investigar cómo la literatura ha resuelto los problemas que usted va a encontrar experimentalmente. Para la construcción del dataset podrá usar el script que se proporciona dentro de la plataforma junto con este set de instrucciones.

Task 1 (Entrega Parcial)

Task 1.1

Implemente el generador y el discriminador siguiendo las reglas de Radford et al. (2015). Los parámetros fijos son: $Z_DIM = 100$, $IMG_SIZE = 64$, $IMG_CHANNELS = 3$, $FEATURES_G = 64$, $FEATURES_D = 64$.

Generador: recibe un vector de ruido $z \in \mathbb{R}^{100}$ con forma (batch, 100, 1, 1) y produce una imagen de forma (batch, 3, 64, 64) mediante cinco capas de ConvTranspose2d con los siguientes canales en orden: $[FEATURES_G*8, FEATURES_G*4, FEATURES_G*2, FEATURES_G, IMG_CHANNELS]$. BatchNorm2d y ReLU después de cada capa excepto la última, que usa Tanh.

Discriminador: recibe una imagen (batch, 3, 64, 64) y produce un escalar por imagen mediante cinco capas de Conv2d con stride 2 y los siguientes canales: $[FEATURES_D, FEATURES_D*2, FEATURES_D*4, FEATURES_D*8, 1]$. BatchNorm2d y LeakyReLU(0.2) después de cada capa excepto la primera y la última. La última usa Sigmoid.

Requerimiento: su implementación debe cumplir las formas exactas. Puede verificarlas con:

```
z = torch.randn(4, Z_DIM, 1, 1)
assert G(z).shape == (4, 3, 64, 64), "Forma del generador incorrecta"
assert D(torch.randn(4, 3, 64, 64)).shape == (4,), "Forma del discriminador incorrecta"
```



Task 1.2

Implemente el entrenamiento alternado. Para cada batch:

Paso del discriminador: calcular la pérdida sobre datos reales con etiqueta 1, calcular la pérdida sobre datos falsos con etiqueta 0 (usando `.detach()` sobre las imágenes del generador para no propagar gradientes hacia G), sumar ambas pérdidas y actualizar θ_D .

Paso del generador: generar un nuevo batch de imágenes falsas, pasarlas por el discriminador, calcular la pérdida usando etiquetas 1 (truco de Goodfellow: el generador quiere que el discriminador clasifique sus salidas como reales), actualizar θ_G .

Use BCELoss como función de pérdida, Adam con $\text{lr}=2\text{e-}4$ y $\text{betas}=(0.5, 0.999)$ para ambos optimizadores (los betas específicos de DCGAN), y $\text{batch_size}=32$.

Entrene por 50 épocas y guarde en cada época: loss_G , loss_D , y una grilla de 16 imágenes generadas con el mismo vector de ruido fijo para visualizar la evolución.

Task 1.3

Produzca dos visualizaciones:

La primera es una grilla de 4×4 imágenes generadas al final del entrenamiento, con los valores desnormalizados al rango $[0, 255]$.

La segunda son las curvas de loss_G y loss_D a lo largo de las 50 épocas en la misma figura. Anote en la figura el punto donde las pérdidas son más cercanas entre sí.

Task 2 (Entrega Final)

Task 2.1

Modifique el balance de entrenamiento para inducir deliberadamente el modo colapso. Específicamente, entrene el discriminador por 5 pasos por cada 1 paso del generador durante 20 épocas.

Muestre visualmente que el modo colapso ocurrió: la grilla de 16 imágenes generadas debe mostrar baja diversidad. Luego responda:

- Explique matemáticamente por qué entrenar D muchas más veces que G induce el modo colapso. Su respuesta debe incluir qué le sucede al gradiente $\nabla_{\theta_G} \mathcal{L}_G$ cuando $D(G(z)) \approx 0$ para todas las muestras, y por qué el generador no puede recuperarse de ese estado.
- En el modo colapso observado, ¿qué valor toma aproximadamente $D^*(x)$ para las imágenes que el generador produce repetidamente? Justifique desde la fórmula del discriminador óptimo derivada en clase.
- Proponga una modificación concreta al loop de entrenamiento (que no sea simplemente cambiar la proporción de pasos) que podría prevenir el modo colapso en su implementación. Justifique por qué funcionaría en términos del gradiente.





Task 2.2

Usando su discriminador entrenado en la Parte 1.2, estime empíricamente la divergencia Jensen-Shannon entre p_{data} y p_G en distintos momentos del entrenamiento.

Recuerde que en clase demostramos:

$$V(D^*, G) = -\log 4 + 2 \cdot \text{JSD}(p_{data} \parallel p_G)$$

Por lo tanto, dado el valor de $V(D, G)$ en cualquier punto del entrenamiento, puede estimar:

$$\hat{\text{JSD}} = \frac{V(D, G) + \log 4}{2}$$

Calcule $\hat{\text{JSD}}$ usando los valores de loss_G y loss_D registrados en cada época y grafique su evolución a lo largo del entrenamiento.

Luego responda:

- El estimado $\hat{\text{JSD}}$ que usted calcula asume que D es el discriminador óptimo D^* para el G actual. En la práctica eso nunca es cierto durante el entrenamiento alternado. Explique en qué dirección sesga ese supuesto su estimado: ¿lo sobreestima o lo subestima? ¿Por qué?
- Hacia el final del entrenamiento, si su GAN ha convergido bien, ¿hacia qué valor debería tender $\hat{\text{JSD}}$? Verifique si su curva empírica es consistente con ese valor teórico.

:

Task 3 (Entrega Final)

Seleccione **uno** de los siguientes problemas de las GANs originales y encuentre el paper que lo resuelve:

- Opción A:** El problema del gradiente que desaparece cuando el discriminador es demasiado bueno. Paper objetivo: WGAN (Wasserstein GAN, Arjovsky et al., 2017).
- Opción B:** La inestabilidad del entrenamiento y la dificultad de evaluar la calidad de las imágenes generadas. Paper objetivo: Inception Score o FID (Heusel et al., 2017).
- Opción C:** El modo colapso y la falta de diversidad en las muestras generadas. Paper objetivo: Unrolled GANs (Metz et al., 2017) o MinibatchGAN.

El paper debe ser de un venue indexado: NeurIPS, ICML, ICLR o CVPR.

Escriba una sección de investigación de 400 a 600 palabras que responda:

- ¿Qué problema específico de las GANs originales identifica el paper y cómo lo formaliza matemáticamente? No es suficiente decir "el modo colapso": debe explicar la formulación matemática del problema tal como aparece en el paper.
- ¿Qué modificación propone el paper respecto a la función de valor $V(D, G)$ o al procedimiento de entrenamiento? Escriba la nueva función objetivo si el paper la propone.



- 
- c) Conecte explícitamente la solución del paper con algo que usted observó experimentalmente en la Parte 2. Si el paper resuelve el gradiente que desaparece y usted lo observó en 2.1, esa conexión debe ser específica: qué valores de gradiente observó y cómo la solución del paper los habría modificado.

Entregas en Canvas

1. Documento PDF con las respuestas a cada task
2. En la entrega parcial se espera que entreguen lo **señalado**, En la entrega final deben entregar **TODOS LOS TASK**
3. Archivo .ipynb, o link a repositorio de GitHub (**No se acepta entregas en otros medios**)
 - a. El código debe estar comentado explicando la relación con las fórmulas de las diapositivas

Evaluación

1. [2.48 pt] Task 1
2. [1.92 pt] Task 2
3. [1.10 pt] Task 3

Total: 5.5 pts